

Summer term 2026

Visual Data Analysis

Assignment Sheet 6

Solution has to be submitted via eCampus by May 22, 2026, 10:00 a.m.

If you have questions concerning the exercises, please use the forum on eCampus.

- Please work on this exercise in **small groups** of 3 students. Submit each solution only once, but clearly indicate who contributed to it by forming a team in eCampus. Remember that all team members have to be able to explain all answers.
- Please submit your answers in PDF format, and your scripts as *.py/*.ipynb files. If you are using [Jupyter notebook](#), please also export your scripts and results as PDF.

Exercise 1 (Working with HSV Color Space, 14 Points)

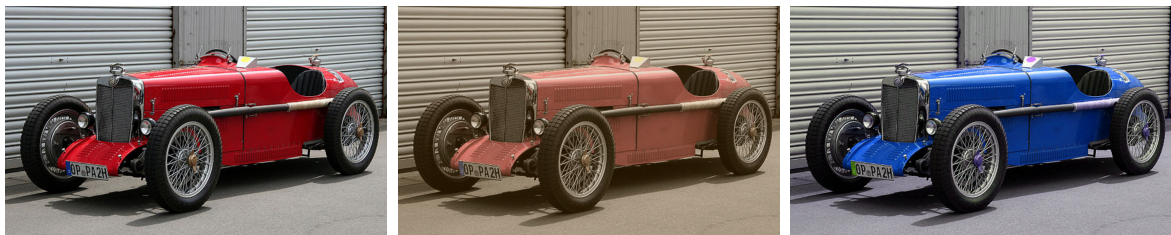


Figure 1: The input image (left), and results of two of the transformations we are asking you to perform on it.

We discussed different color spaces in the lecture. In this exercise, you will use them to perform different operations on the color image `oldtimer.png` which is provided on eCampus. Please use Python for your solution. It is recommended to use routines from `matplotlib`, which allows you to do most subtasks in very few and simple lines of code. However, *do not* use an image processing program like Photoshop. Your tasks are as follows:

- a) Read the given `oldtimer.png` image. Convert it to HSV color space, and create a grayscale version by retaining only the value. (3P)
- b) Think about another way to convert color image to grayscale. Its result should look reasonable (with bright regions appearing bright), but it should differ from the one you obtained from the HSV color space. Implement and briefly justify your method. (3P)
- c) Output a version of the image in which saturation has been reduced by 50%. (2P)
- d) Create a simple “aged photograph” effect by globally blending (via simple weighted averaging or some other [blend mode](#) of your choice) the half-desaturated image from the previous step with some amount of brown. (2P)
- e) Rotate the hues of the original image so that the car obtains a different color. Provide two images in which the car is blue, and has another color of your choice. (4P)

Exercise 2 (Lightness in Color Maps, 12 Points)

When color mapping ordered quantities, it is a widely used design rule to monotonically vary lightness. But is it more natural to map the highest value to the darkest color or to the lightest one? Should that decision depend on the background on which the plot is presented? Or is it even more complicated? The paper [schloss-color-to-meaning-2019.pdf](#), which you can find on eCampus, investigates these questions with an empirical study. As always, please answer the following questions about it in your own words.

- a) A straightforward way to find out if test subjects intuitively interpret darker or lighter colors as representing higher values would be to ask them to interpret plots without a legend. Why do the authors instead decide to show legends and evaluate response times? (2P)
- b) When interpreting visualizations, accuracy should be at least as important as response times. How do the authors account for that in their experiments? (2P)
- c) Authors observe that neither the hypothesis of a “dark-is-more” bias nor a “contrast-is-more” bias could convincingly explain all available experimental results. What explanation do they propose instead? (2P)
- d) Do all experimental results in the paper agree with their proposed explanation? If not, what is the exception and what do they mention as a potential reason for it? (2P)
- e) Authors propose an “Opacity Variation Index” whose definition relies on the CIELAB color space. Do you see a reason for preferring CIELAB over alternatives such as RGB or XYZ for this purpose? (2P)
- f) Several plots include error bars that represent standard errors of the means. Please explain how that differs from the standard deviation of the measurements. Use additional sources of information if needed. (2P)

Exercise 3 (Running Time of Reingold-Tilford Algorithm, 9 Points)

The Reingold-Tilford algorithm not only provides tidy tree layouts, but is also computationally efficient. In this exercise, you will consider how the idea of threading that was mentioned in the lecture reduces the time complexity of computing the required horizontal shifts, and how we can reduce the complexity of applying those shifts.

- a) Computing the correct separation between siblings requires traversing the contours of the respective subtrees. If subtrees have heights h_L and h_R , respectively, what is the complexity of computing the separation by tracing the inner contours? What is the complexity of introducing the new thread (if needed) by additionally tracing the outer contours as well? (2P)
- b) Why can any node be visited at most once as part of an inner contour comparison throughout the entire execution of the algorithm? What does this imply with respect to the overall complexity of all separation computations? (2P)
- c) Assume that, whenever you place a parent node, you shift all nodes in its subtrees to avoid overlaps, by recursively updating their horizontal coordinate. What is the asymptotic running time for shifting a subtree with n_s nodes, and the worst case asymptotic running time for all shifts in the overall algorithm? (2P)
- d) In the implementation proposed by Reingold and Tilford, instead of storing absolute positions, each node initially stores its relative position with respect to its parent as an offset. Describe how one would shift all nodes in a subtree in that case, and state the complexity for a subtree with n_s nodes. Also describe in words or as pseudo code how one would assign final absolute positions based on those offsets. What is the time complexity of that process? (3P)

Exercise 4 (Graph Visualization in Python, 15 Points)

In this exercise you will learn how to visualize graphs in Python. We suggest to further use `plotly` and the `Networkx` package, which was used for our own example solution. In case you should already know and prefer an alternative Python package that allows you to perform the same tasks, you are free to use it.

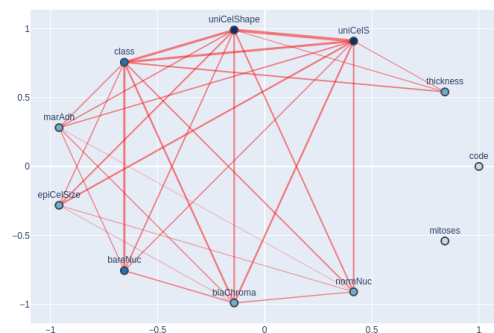


Figure 2: Variable correlation graph.

- Read the [Breast Cancer Dataset](#) `breast-cancer-wisconsin.xlsx` (from Sheet 3) and fill in the missing values as before. Then compute the Pearson correlation between any pair of variables, and store them in a matrix. (2P)
- Create a graph from the correlation matrix and visualize it with a force-directed layout. Represent each variable as a node in the graph. Insert an edge between two variables whenever the Pearson correlation between them exceeds the threshold $\rho > 0.6$. Tune the hyperparameters of the force-directed layout, such that the disconnected nodes do not drift too far away. (5P)
- Modify the visual attributes of edges to reflect the magnitude of the correlation. (2P)
- Produce an alternative visualization with a circular layout. Color the nodes so that there are four sets of nodes, one color for having at least one correlation more than 0.9 to other nodes, another for having at least a correlation $0.8 < \rho_{\max} \leq 0.9$, one for having a correlation $0.6 < \rho_{\max} \leq 0.8$ and the last one for the remaining nodes. (3P)
- Answer the following questions:
 - At the selected threshold, which nodes are disconnected from the rest of the graph and what do they indicate? (1P)
 - If two nodes A and B are strongly correlated, and node C is strongly correlated with node B, can we conclude that node C will be also strongly correlated with node A? (1P)
 - Based on the visualization, which variables would you propose to predict the class? (1P)

Good Luck!